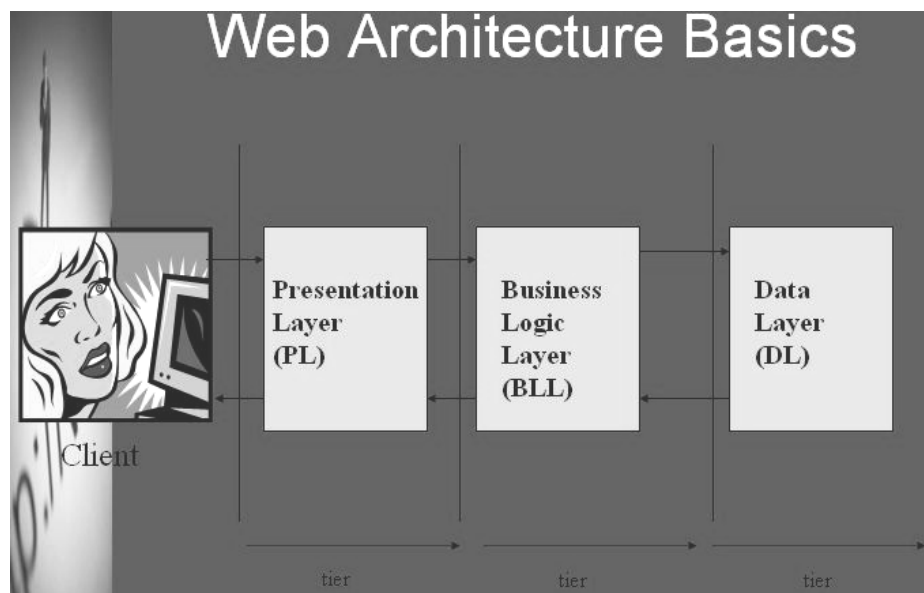


บทที่ 2

ทฤษฎีพื้นฐานที่ใช้ในการวิจัย

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานต่างๆที่เกี่ยวข้องในการวิจัยซึ่งประกอบด้วยสถาปัตยกรรมของเว็บ, เชนที่ทีพี โพรโทคอล (HTTP Protocol), เว็บเซิร์ฟเวอร์ (Web Server), ดาต้าเบสเซิร์ฟเวอร์ (Database Server), ภาษาที่ใช้ในการเขียนเว็บ, ยูอาร์แอล เอ็นโค้ดดิ้ง (URL Encoding) ซึ่งเนื้อหาทั้งหมดนี้จำเป็นสำหรับการศึกษาระบบผ่านเว็บ

2.1 สถาปัตยกรรมของเว็บ



รูปที่2.1 สถาปัตยกรรมของเว็บ

สถาปัตยกรรมของเว็บโดยทั่วไปประกอบด้วยสามส่วนดังนี้

1. Presentation Layer (PL)

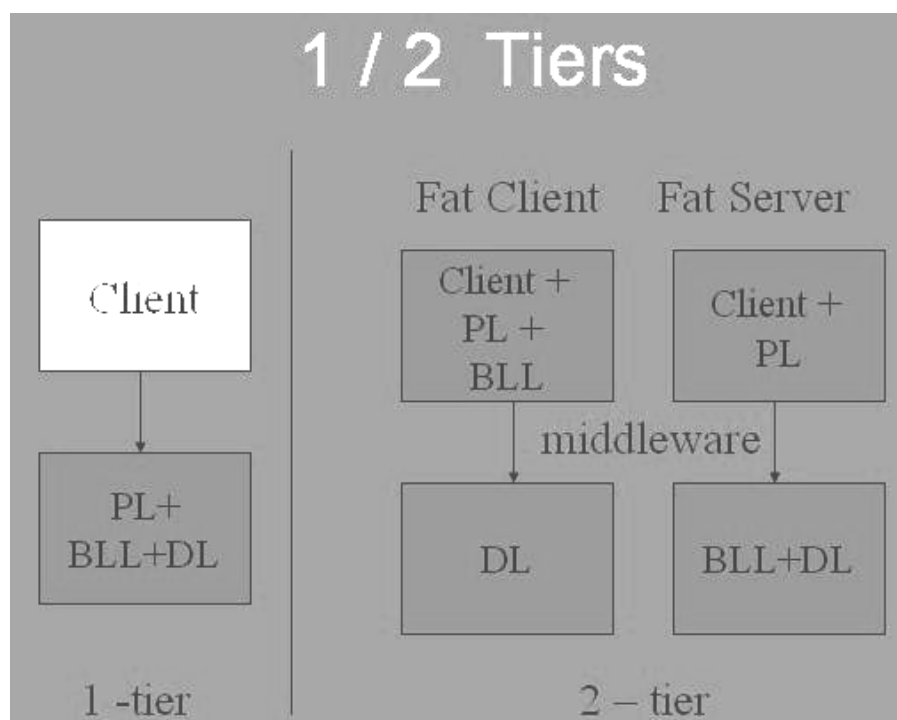
เป็นชั้นที่อยู่ด้านนอกสุดซึ่งเมื่อเครื่องลูกข่ายติดต่อมาส่วนนี้จะทำหน้าที่รับการร้องขอมาก่อนแล้วกระทำไปตามการร้องขอที่ขอมาคือเป็นตัวจัดการเกี่ยวกับการร้องขอของเครื่องลูกข่ายทั้งหมด

2. Business Logic Layer (BLL)

เป็นชั้นที่เป็นส่วนจัดการเกี่ยวกับโปรเซสทางด้านธุรกิจซึ่งเป็นตามรูปแบบของธุรกิจนั้นๆ ในชั้นนี้จึงไม่เหมือนกันในแต่ละธุรกิจอาจต่างกันหรือเหมือนกันก็ได้

3. Data Layer (DL)

เป็นชั้นที่ใช้ของข้อมูลที่มีการใช้กายเว็บ ซึ่งชั้นนี้มักจะมีความปลอดภัยที่สุดเพราะอยู่ข้างหลังสุดและสำคัญที่สุดเพราะข้อมูลต่าง ๆ นั้นเป็นสิ่งที่สำคัญเป็นอันดับหนึ่ง



รูปที่ 2.2 สถาปัตยกรรม 1-tier และ 2 tier

2.2.1 1-tier

เป็นการรวมส่วนของ ฟรอนต์เอนด์เลเยอร์ (Presentation Layer: PL), บิสซิเนสลอจิกเลเยอร์ (Business Logic Layer: BLL), ดาต้าเลเยอร์ (Data Layer: DL) เข้าเป็นชั้นเดียวกัน ซึ่งทำให้มีความปลอดภัยค่อนข้างต่ำเนื่องจากหากโดนเจาะระบบได้ก็จะได้ก็เหมือนกับการได้ทุกอย่างไป

2.1.2 2-tier

แบ่งเป็น 2 ชั้น โดยแยกเอาดาต้าเลเยอร์ (Data Layer: DL) ออกมาเป็นอีกชั้นหนึ่งต่างหาก ซึ่ง 2-tier นั้นแบ่งเป็นสองประเภทดังนี้

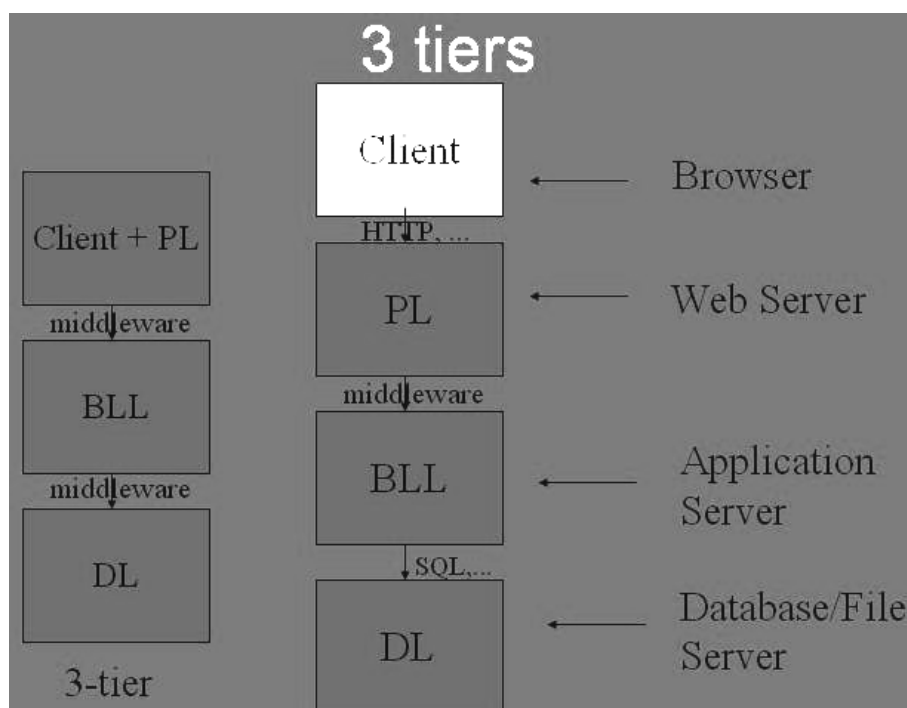
- Fat Client
- Fat Server

Fat Client คือให้ทางเครื่องลูกข่ายเป็นฝั่งประมวลในชั้นของ BLL ด้วยทำให้ลดภาระของเครื่องผู้ให้บริการไปได้แต่เครื่องลูกข่าย ต้องมีประสิทธิภาพที่ดีพอที่จะประมวลผลได้

Fat Server คือส่วนของ BLL นั้นถูกประมวลผลโดยเครื่องผู้ให้บริการ ซึ่งเครื่องลูกข่ายไม่ต้องมีประสิทธิภาพสูงก็ได้ แต่ จะทำให้เครื่องผู้ให้บริการนั้นทำงานหนักขึ้น

2.1.3 3-tier

มีการแยกแต่ละส่วนออกจากกัน คือ ฟรอนต์เอนด์เลเยอร์(Presentation Layer: PL), บิสซิเนสลอจิกเลเยอร์ (Business Logic Layer: BLL), ดาต้าเลเยอร์ (Data Layer: DL) ดังรูปที่ 2.3

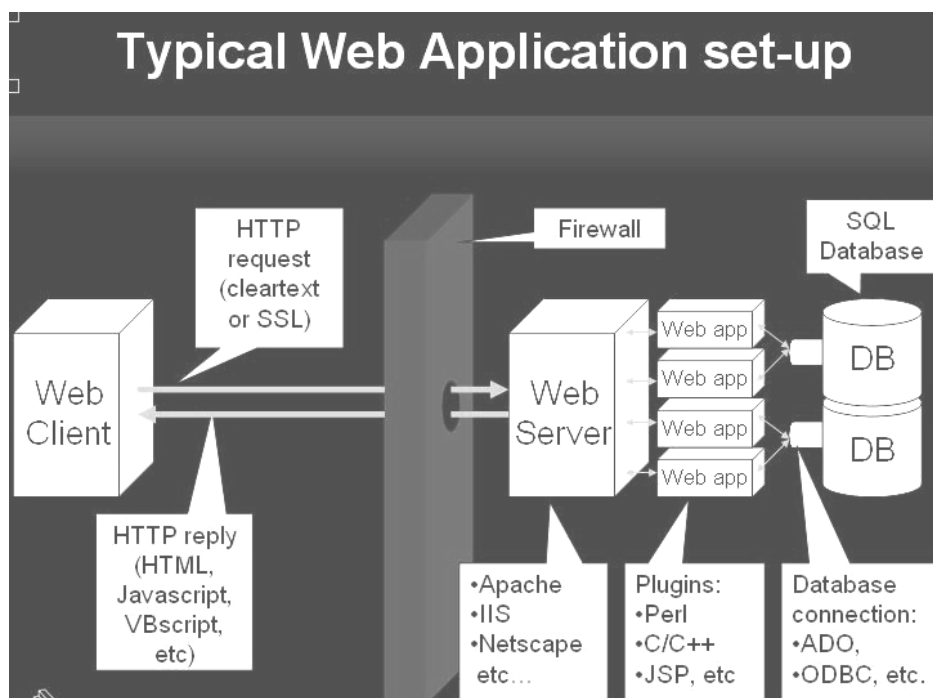


รูปที่ 2.3 สถาปัตยกรรม 3-tier

ซึ่งในระดับปฏิบัติการนั้น ชั้นต่างๆที่กล่าวในตอนต้นนั้นจะเป็นดังรูปข้างล่างนี้

ชั้นฟรอนต์เอนด์เลเยอร์(Presentation Layer: PL) นั้นในระดับการปฏิบัติการนั้นจะเป็นเว็บเซิร์ฟเวอร์ ที่คอยจัดการสิ่งต่างๆให้กับเครื่องลูกข่าย ซึ่งเครื่องลูกข่ายจะติดต่อกับเว็บเซิร์ฟเวอร์โดยผ่านตัวกลางที่ใช้ในการติดต่อหรือที่เรียกว่า middleware ซึ่งจะเป็นสิ่งที่ใช้ติดต่อระหว่างชั้นแต่ละชั้นก็จะมี middleware ไม่เหมือนกันใน ส่วนชั้น บิสซิเนสลอจิกเลเยอร์ (Business Logic Layer: BLL) จะเป็นเว็บแอปพลิเคชันเซิร์ฟเวอร์ และชั้นสุดท้ายจะเป็นชั้นดาต้าเลเยอร์ (Data Layer: DL) จะเป็น Database/File Server

ซึ่งโดยทั่วไปการติดตั้งและการติดต่อ เว็บแอปพลิเคชันจะเป็นไปตามรูปข้างล่างนี้



รูปที่2.4 แสดงรูปแบบของการติดตั้งเว็บแอปพลิเคชัน

ตามรูปที่2.4 เป็นการเรียงลำดับการใช้งานเว็บแอปพลิเคชันซึ่งเป็นรูปแบบ 3-tier โดยเริ่มจากเครื่องลูกข่ายทำการร้องขอผ่านทางโปรโตคอล HTTP ซึ่งเป็นการร้องขอทรัพยากรที่เป็นลักษณะ clear text คือไม่มีการเข้ารหัส หรือการใช้โปรโตคอล HTTPS ก็คือโปรโตคอล HTTP ที่วิ่งบน เอสเอสแอล(SSL) ที่มีการเข้ารหัสเอาไว้ทำให้ปลอดภัยมากยิ่งขึ้น เมื่อการร้องขอผ่านมาทาง พอร์ต80 หรือ พอร์ต443 ที่เป็น HTTP และ HTTPS ตามลำดับก็จะสามารถผ่านไฟร์วอลล์ไปได้ไปยังเว็บเซิร์ฟเวอร์ซึ่งเว็บเซิร์ฟเวอร์อาจจะเป็น Apache, IIS, Netscape หรือ เว็บเซิร์ฟเวอร์อื่นก็ได้ จากนั้นจะส่งผ่านไปยัง Web Application ซึ่งอาจจะเขียนด้วย Perl, PHP, JSP หรือภาษาอื่นๆก็ได้ เมื่อมาถึง เว็บแอปพลิเคชันถ้ามีการติดต่อกับดาต้าเบส ก็จะไปติดต่อกับ Data base/File Server ซึ่งการติดต่อจากเว็บแอปพลิเคชันไปยัง Data base/File Server อาจจะใช้การติดต่อแบบ ODBC,ADO หรือการติดต่อแบบอื่นๆ ซึ่งเมื่อได้ทรัพยากรที่เครื่องลูกข่ายร้องขอแล้วเว็บเซิร์ฟเวอร์ก็จะทำการตอบสนองการร้องขอ ด้วย HTTP reply ซึ่งจะประกอบด้วย HTML, JavaScript, VBscript หรือ อื่นๆ

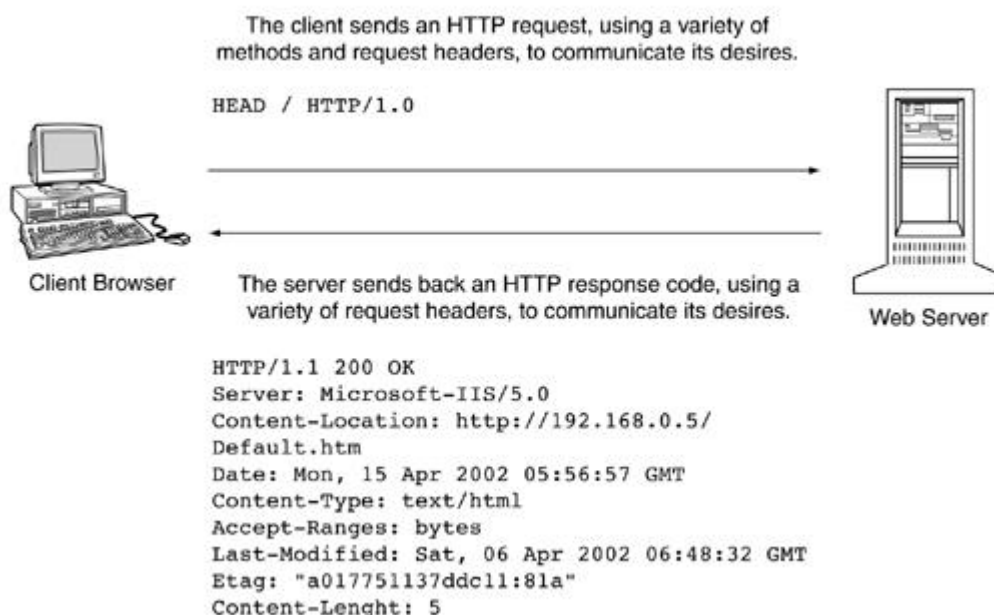
2.2 HTTP Protocol

2.2.1 HTTP

HTTP เป็นโปรโตคอลที่ได้รับความนิยมแพร่หลายที่สุดบนอินเทอร์เน็ต ทุกๆเบราว์เซอร์และเซิร์ฟเวอร์จำเป็นต้องสื่อสารกันโดยใช้โปรโตคอลเหล่านี้ เพื่อแลกเปลี่ยนข้อมูลโปรโตคอลนี้มีอยู่ด้วยกัน 3 เวอร์ชัน ซึ่งทุกเวอร์ชันล้วนมียังคงใช้งานโครงสร้างพื้นฐานเดิม HTTP นี้เป็นโปรโตคอลสำหรับการร้องขอ (request) และให้การตอบสนอง (response) ที่ไม่มีการจดจำสถานการณ์ทำงาน (stateless protocol) ซึ่งเปิดโอกาสให้เครื่องคอมพิวเตอร์สามารถติดต่อกับเครื่องอื่นๆได้อย่างค่อนข้างมีประสิทธิภาพและก่อให้เกิดการสนทนาพูดคุยกันทุกชั่วโมง ทุกวัน และทุกสัปดาห์

ถึงแม้ข้อกำหนดของ HTTP/1.0 ซึ่งกำลังถูกใช้งานอยู่ในปัจจุบันจะไม่เหมือนกันทีเดียวนักกับข้อกำหนดดั้งเดิมที่คิดค้นโดย Tim Berners-Lee ในเดือนมีนาคม ปี 1990 แต่ฟีเจอร์การทำงานพื้นฐานต่างๆของ HTTP ก็ยังอ้างอิงตามข้อกำหนดดั้งเดิม รูปข้างล่างนี้ เน้นให้เห็นถึงองค์ประกอบหลักของโปรโตคอล HTTP และการใช้งานของมัน

Figure 4-1. The HTTP protocol



รูปที่ 2.5 รูปแบบการติดต่อผ่าน HTTP Protocol

2.2.2 HTTP/0.9

ข้อกำหนดอย่างเป็นทางการแรกของ HTTP โดยทั่วไปจะถูกพิจารณาว่าคือข้อกำหนดของ HTTP/0.9 เวอร์ชันนี้และเวอร์ชันถัดมาได้รับการนิยามไว้ใน Request for Comments (RFC) ของ Internet Engineering Task Force (IETF) หมายเลข 1945 (<http://www.ietf.org/rfc/rfc1945.txt>) เป็นเวลาประมาณ 4 ปี (1992-1996) ที่ HTTP/0.9 ได้วางพื้นฐานให้กับโลกของอินเทอร์เน็ตมาตั้งแต่สมัยที่เว็บยังไม่ได้ฟูเฟื่องขนาดนี้ HTTP/0.9 นั้นยังคงมีข้อจำกัดอยู่หลายอย่างและไม่ครอบคลุมสิ่งที่เราพิจารณาว่าเป็นองค์ประกอบสำคัญในโลกของเว็บที่มีการโต้ตอบกันมากขึ้นอย่างในปัจจุบัน

2.2.3 HTTP/1.0

ข้อกำหนดของ HTTP/1.0 นั้นเริ่มถือกำเนิดมาพร้อมกับช่วงที่อินเทอร์เน็ตนั้นเริ่มร้อนแรงขึ้น ถึงแม้อายุของมันจะยังไม่มากนักในแง่ของการเติบโตทางเทคโนโลยี โดยที่มันเพิ่งได้รับการสรุปมาตรฐานครั้งสุดท้ายเมื่อเดือนพฤษภาคม ปี ค.ศ. 1996 แต่ HTTP/1.0 ก็ยังคงถือว่าเป็นเจ้าแห่งโปรโตคอล HTTP บนอินเทอร์เน็ต เว็บเซิร์ฟเวอร์ส่วนใหญ่และบราวเซอร์ยังคงใช้งาน HTTP/1.0 อยู่สำหรับการสื่อสารปกติ เช่นเดียวกับ HTTP/0.9, HTTP/1.0 ได้รับการอธิบายไว้ใน RFC 1945

ฐานรากหลักหรือกลไกการทำงานหลักของโปรโตคอล HTTP/1.0 อยู่กับการแลกเปลี่ยนการร้องขอ/การตอบสนองกลับ (request/response) การแลกเปลี่ยนนี้จะช่วยให้เกิดการส่งข้อมูลไป ตีความและส่งข้อมูลกลับระหว่างไคลเอนต์ (เว็บบราวเซอร์) และเซิร์ฟเวอร์ (เว็บเซิร์ฟเวอร์) หรือหยุดการส่งข้อมูล

โดยทั่วไป, URL ที่ใช้ใน HTTP/1.0 จะคล้ายด้านล่างนี้

(2.1)

`http://host [ ":" port ] [ absolute_path ]`

Host หมายถึงชื่อโฮสต์ที่ต้องการ และ port หมายถึงการระบุหมายเลขพอร์ตปลายทาง และ absolute_path คือทรัพยากรที่ต้องการ

2.2.4 HTTP/1.1

HTTP/1.1 ซึ่งได้รับการประกาศอย่างเป็นทางการในปี 2001 เป็นโปรโตคอล HTTP ที่ได้รับการพัฒนามาที่สุดและได้รับการใช้งานอย่างกว้าง RFC 2616 ของ IETF ได้กล่าวถึงรายละเอียดเฉพาะของเวอร์ชันล่าสุดนี้และเน้นถึงฟังก์ชันการทำงานเพิ่มเติมที่เพิ่มขึ้นจากเวอร์ชัน HTTP/1.0 สิ่งสำคัญหลักที่ขาดหายไป ใน HTTP/1.0 และก่อให้เกิดความจำเป็นในการมีเวอร์ชัน 1.1 ได้แก่

การสนับสนุน HTTP proxy การสนับสนุนเรื่องเว็บแคชที่มีน้อยมากและไม่มีการจัดการกับคอนเนกชันถาวร และเวอร์ชันโฮสต์ที่เหมาะสม

URL ของ HTTP/1.1 จะคล้ายด้านล่างนี้

(2.2)

```
http://host [ ":" port ] [ absolute_path [ "?" query ] ]
```

แม้จะคล้ายกันกับ URL ของ HTTP/1.1 จะแตกต่างกันอย่างเห็นได้ชัดตรงที่ มันสนับสนุนการส่งผ่านพารามิเตอร์ของสคริปต์ด้วยเครื่องหมาย “?” เครื่องหมายนี้เป็นหัวใจสำคัญของเว็บแอปพลิเคชันหลักทั้งหมดและถือเป็นหนึ่งในอาวุธสำคัญสำหรับการโจมตี สิ่งใดๆ ที่ถูกระบุไว้หลังจากเครื่องหมาย “?” ถือเป็นคอนเทนต์ที่สคริปต์ต้องประมวลผล ดังนั้นจึงเป็นเป้าหมายของการนำมาใช้โจมตี

2.2.5 HTTP Request

ขั้นตอนแรกของการร้องขอ ก็คือ การตัดสินใจเลือกเมธอด หรือวิธีการใช้งาน ตารางที่ 2.1 ข้างล่างนี้ แสดงรายละเอียดของเมธอดของโปรโตคอล HTTP/1.0

ตารางที่ 2.1 รายละเอียดของเมธอดของโปรโตคอล HTTP/1.0

Method	คำอธิบาย
GET	ดึงเอาข้อมูลที่ต้องการจากระบบไฟล์บนเซิร์ฟเวอร์ ถ้าไฟล์ที่ต้องการเป็นไฟล์ HTML แบบสแตติก คอนเทนต์หรือเนื้อหาภายในไฟล์จะถูกแสดงขึ้นมา อย่างไรก็ตาม ถ้าไฟล์นั้นเป็นไฟล์ ASP แบบไดนามิก เว็บเซิร์ฟเวอร์ก็ต้องประมวลผลไฟล์นั้นก่อน หรือเอ็กคิวต์คำสั่งและส่งเอาผลลัพธ์ของคำสั่งเหล่านี้ไปให้เบราว์เซอร์ที่ร้องขอ
HEAD	เมธอด HEAD นั้นคล้ายกันมากกับเมธอด GET ต่างกันอยู่เพียงอย่างเดียวก็คือ มันจะไม่ส่งข้อมูลที่ต้องการกลับมาให้ อย่างไรก็ตาม จุดเด่นของ HEAD ก็คือ มันจะตอบสนองกลับมาด้วยข้อมูลดิบ (meta information) เช่น โค้ดตอบสนองจากเซิร์ฟเวอร์ เซคเตอร์ แสดงวันที่ และอื่นๆ
POST	เมธอด POST จะร้องขอให้เซิร์ฟเวอร์ยอมรับข้อมูลที่ส่งไปและเอ็กคิวต์มัน ส่วนใหญ่เมธอด POST จะถูกนำมาใช้เมื่อเซิร์ฟเวอร์ต้องเกี่ยวข้องกับ CGI หรือ server-side scripting

2.2.6 HTTP Response

HTTP request จากเครื่องไคลเอนต์จะได้รับการจัดการโดยเซิร์ฟเวอร์และมีการตอบสนองกลับอย่างเหมาะสม ในการตอบสนอง เซิร์ฟเวอร์จะส่งกลับเมสเสจซึ่งประกอบด้วยชุดของฟิลด์ประเภทเหล่านี้มาให้

- โค้ดตอบสนองเป็นตัวเลขที่บ่งชี้ให้ทราบถึงพฤติกรรมการตอบสนองกลับของเว็บเซิร์ฟเวอร์
- ฟิลด์เฮดเดอร์ข้อมูลเพิ่มเติมเกี่ยวกับการตอบสนองกลับ
- ข้อมูล คอนเทนต์หรือเนื้อหาภายในเมสเสจที่ตอบสนองกลับมา

ด้วยชุดของฟิลด์ข้างต้น ไคลเอนต์เบราว์เซอร์จะเข้าใจถึงสิ่งที่เซิร์ฟเวอร์ตอบสนองกลับมาและทำงานโต้ตอบกับเซิร์ฟเวอร์ได้อย่างเหมาะสม

2.3 Web Server

2.3.1 Web Server

ทุกครั้งที่เราเบราว์เซอร์ทำการเชื่อมต่อเว็บไซต์บนอินเทอร์เน็ต (หรือบนอินทราเน็ต แล้วแต่กรณี) เซิร์ฟเวอร์จะคอยรับฟังการร้องขอนั้นบนเน็ตเวิร์ก และตอบสนองต่อผู้ร้องขอด้วยข้อมูลที่ต้องการ

2.3.2 Apache

ดังที่ได้กล่าวไว้ในเว็บไซต์ของ Apache Software Foundation ว่า “Apache เป็นเว็บเซิร์ฟเวอร์ที่ได้รับความนิยมมากที่สุดบนอินเทอร์เน็ตตั้งแต่เดือนเมษายน ปี 1996” สาเหตุสำคัญของการได้รับความนิยมมีอยู่ด้วยกัน 3 ประการ ได้แก่ การทำงานได้บนหลายแพลตฟอร์ม ฟิเจอร์ที่มีมากและราคาต้นทุนที่ต่ำ Apache Web Server ทำงานได้บนเกือบทุกแพลตฟอร์มที่มีอยู่ในปัจจุบัน ได้แก่ NetBSD, Digital UNIX, AIX, OS/2, Windows 3.x, SCO, HP-UX, Novell NetWare, Macintosh, Be OS, Windows NT, Linux, VMS, AS/400, Windows 95, FreeBSD, IRIX และ Solaris นอกจากนี้ Apache Web Server ยังได้ให้ฟิเจอร์ที่หลากหลายเพื่อให้นักพัฒนาสามารถสร้างและเพิ่มขยายขีดความสามารถในการออกแบบเว็บได้อย่างง่ายดายและรวดเร็วและท้ายที่สุดที่สำคัญก็คือ Apache Web Server ได้ในราคาที่ต่ำที่สุด นั่นก็คือ มันฟรี!

กว่า 56 % ของเว็บเซิร์ฟเวอร์ทั้งหมดในปัจจุบัน ใช้งาน Apache (สำรวจโดย Netcraft 2001) มันจึงไม่น่าแปลกใจเลยว่าทำไมมันจึงเป็นเป้าหมายใหญ่ของบรรดาแฮกเกอร์ทั้งหลาย ทุกๆ แพลตฟอร์มและทุกๆ ฟิเจอร์ที่เพิ่มเข้าไปล้วนแต่ให้ออกาสแฮกเกอร์ในการฉวยโอกาสจากช่องโหว่ด้านการโปรแกรมของฟิเจอร์เหล่านั้นเพื่อการโจมตี

เวอร์ชันของ Apache ที่เรากล่าวในบทนี้คือเวอร์ชัน 2.0.32 มันมีฟิเจอร์ที่เราควรตระหนักในด้านความปลอดภัย

- เวอร์ชวลโฮสต์ (virtual hosts)
- Server Side Includes
- การสร้างไดนามิกคอนเท้นต์ด้วย CGI
- แชนเดล (handler)
- ตัวแปรสถานะแวดล้อม (environment variables)
- การแมป URL เข้ากับระบบไฟล์ (<http://httpd.apache.org/docs-2.0/>)

ฟีเจอร์มากมายที่น่าสนใจของ Apache เป็นตัวดึงดูดทั้งในแง่การนำไปใช้งานที่ดีและในแง่ของการเป็นเป้าหมายต่อการโจมตีแฮกเกอร์ส่วนใหญ่ที่กำลังพยายามในการค้นหาช่องโหว่ของมันและฉวยโอกาสจากช่องโหว่นั้น

2.3.3 Internet Information Server (IIS)

ไมโครซอฟต์ได้พัฒนา IIS Web Server ขึ้นมาเพื่อส่งเสริมกลยุทธ์ด้านอินเทอร์เน็ตของตน และเพื่อให้องค์กรต่างๆ ใช้งานมันจึงได้ผนวกรวมเอา IIS เข้ามาเป็นเว็บเซิร์ฟเวอร์หนึ่งในระบบปฏิบัติการวินโดวส์ด้วยตั้งแต่ NT 4.0 ถึงแม้ IIS จะมีส่วนแบ่งการตลาดไม่สูงเมื่อเทียบกับ Apache แต่มันก็เป็นซอฟต์แวร์ที่มีจุดเด่นหลายอย่างอยู่ในตัว แต่ทว่าค่าดีฟอลต์คอนฟิกูเรชันของ IIS นั้นส่วนใหญ่แล้วเป็นบ่อเกิดของช่องโหว่ด้านความปลอดภัยทั้งสิ้นในหัวข้อนี้ เราจะกล่าวถึงองค์ประกอบด้านความปลอดภัยของ IIS และช่องโหว่พื้นฐานที่เปิดโอกาสให้แฮกเกอร์ทำให้ไฟร์วอลล์ของคุณกลายเป็นเพียงแค่นกหวีดเท่านั้นเอง

ISAPI Applications เมื่อพวกเราได้รับคำถามว่า “อะไรคือความเสี่ยงด้านความปลอดภัยที่ใหญ่หลวงที่สุดของเว็บเซิร์ฟเวอร์ IIS?” คำตอบที่แท้จริงก็คือ “Internet Server Application Programming Interface (ISAPI) Application” ISAPI ทำให้นักพัฒนาเว็บเพิ่มขยายขีดความสามารถของเว็บเซิร์ฟเวอร์ IIS ได้ด้วยการเปิดโอกาสให้พวกเขาเขียนโปรแกรมของตนขึ้นมาเองเพื่อประมวลผลและจัดการกับข้อมูลและการร้องขอต่างๆ ที่ส่งมายังเว็บเซิร์ฟเวอร์

นอกจากนั้น ISAPI Application ยังอนุญาตให้นักพัฒนาดักจับทุกแพ็กเก็ตที่วิ่งเข้ามาหาเว็บเซิร์ฟเวอร์และประมวลผลล่วงหน้า (pre-process) ก่อน นี่ดูเหมือนว่าจะเป็นเรื่องดี แต่ทว่าจริงๆ แล้ว การประมวลผลแพ็กเก็ตก่อนล่วงหน้า (pre-process) กลับนำไปสู่ช่องโหว่ต่อการโจมตีโดยดีฟอลต์แล้ว, IIS ติดตั้ง ISAPI filters ไว้เป็นจำนวนมากที่สามารถถูกนำมาใช้เพื่อเจาะระบบเว็บเซิร์ฟเวอร์ IIS ได้ ยิ่งเปิดช่องทางอำนวยความสะดวกให้กับนักพัฒนาไว้มากเท่าไร ก็ยิ่งเปิดโอกาสให้แฮกเกอร์ค้นหาช่องทางได้ง่ายขึ้น และ ISAPI filter ก็ให้ช่องทางดังกล่าวนั้นด้วย

ถึงแม้ว่า ISAPI filter จะเพิ่มฟังก์ชันการใช้งานที่มากมายให้กับเว็บเซิร์ฟเวอร์ แต่พวกมันก็เพิ่มภาระความน่าปวดหัวให้กับผู้ดูแลระบบไม่ใช่น้อย ตัวอย่างเช่น ฟิลเตอร์ .idq และ .ida เป็นต้น พวกมันมีส่วนต้องรับผิดชอบต่อความเสียหายจาก Code Red และ Nimda Worm ซึ่งมีมูลค่าความเสียหายกว่า \$100,000 ที่เกิดขึ้นกับบริษัทชั้นนำ 1000 อันดับต้นของอเมริกา Code Red

Worm อาศัยช่องโหว่จากเงื่อนไขการเกิดบัฟเฟอร์โอเวอร์โฟลว์ในส่วนขยาย .ida โดยมันจะฉวยโอกาสจากช่องโหว่ของ .ida ดังกล่าวที่มีอยู่บน Index Server ซึ่งจะเปิดโอกาสให้แฮกเกอร์ส่งการร้องขอ (request) ไปยังเว็บเซิร์ฟเวอร์และกระตุ้นให้เกิดโอเวอร์โฟลว์บน DDL ที่ทำหน้าที่ประมวลผลการร้องขอนั้น และนำไปสู่การเอ็กซีคิวต์คำสั่งใดๆก็ได้บนเซิร์ฟเวอร์ สิ่งที่น่าทึ่งของการโจมตีนี้อยู่ที่ HTTP request ที่ใช้สำหรับการโจมตีนั้นถือเป็นส่วน Payload ธรรมดาของแพ็คเกจ ทำให้ไฟร์วอลล์หรือฟร็อกซิพิจารณาว่ามันเป็น HTTP request ธรรมดาๆ อันหนึ่งที่ใช้งานอยู่ปกติ ผลแห่งการโจมตีนั้นกว้างไกลและรุนแรงยิ่ง นั่นก็คือ แฮกเกอร์สามารถเอ็กซีคิวต์คำสั่งใดๆก็ได้บนเซิร์ฟเวอร์และเข้าครอบครองเครื่องนั้นโดยสมบูรณ์

ทางออกของปัญหาที่เกิดจากฟิลเตอร์นี้ก็คือ ให้ติดตั้งซอฟต์แวร์แพตช์ (Patch) ที่ไมโครซอฟต์ประกาศออกมา อย่างไรก็ตามวิธีการขจัดปัญหาที่ดีที่สุดก็คือ การยกเลิก ISAPI filter ออกจากเว็บเซิร์ฟเวอร์ ซึ่งสามารถกระทำได้โดยใช้ Internet Services Manager ให้เลือกที่ Properties ของเว็บเซิร์ฟเวอร์ที่กำลังทำงานอยู่ จากนั้นเลือกแท็บ ISAPI filter และยกเลิกฟิลเตอร์ที่ไม่จำเป็นออกไป รูปที่ 2.1 ได้แสดงดีฟอลต์ฟิลเตอร์และ application mapping ไว้ ขั้นตอนท้าย ให้ยกเลิกมันออกไปทีละตัว

ISAPI filter เป็นโปรแกรมที่ทำหน้าที่เป็นเสมือนตัวกรองเบื้องต้นที่คอยดักจับการร้องขอ (request) ก่อนที่ ISAPI application ที่อยู่ด้านหลังจะประมวลผลการร้องขอนั้น ISAPI filter โดยดีฟอลต์ที่สามารถยกเลิกหรือดิสเอเบิล ได้แก่

- Sspifilt
- Compression
- Md5filt
- Fpexedll.dll

2.4 Database Server

หัวใจหลักประการหนึ่งของทุกๆเว็บแอปพลิเคชันก็คือ ดาต้าเบสเซิร์ฟเวอร์หรือเซิร์ฟเวอร์ที่จัดการฐานข้อมูลซึ่งทำหน้าที่จัดเก็บและเรียกคืนข้อมูลให้สำหรับทั้งลูกค้าและแอสเกอร์ ข้อมูลภายในฐานข้อมูลเป็นข้อมูลสำคัญของลูกค้าและข้อมูลภายในองค์กร โชคไม่ดี ที่ความปลอดภัยนั้นบางครั้งถูกมองว่าเป็นสิ่งไม่สำคัญนักสำหรับฐานข้อมูล ดังนั้น ช่องโหว่จึงมีอยู่มากมายในฐานข้อมูลที่ได้รับการติดตั้งโดยดีฟอลต์ และอาจมีช่องโหว่อีกมากมายที่ถูกค้นพบในอีกต่อไป

คำสั่งภาษา SQL

ตารางที่2.2 คำสั่ง SQL

คำสั่ง	คำอธิบาย
ALTER DATABASE	แก้ไขฐานข้อมูลที่ต้องการโดยการเพิ่มหรือลบไฟล์
ALTER TABLE	แก้ไขตารางในฐานข้อมูลโดยการเปลี่ยนแปลงเพิ่มเติมหรือลบคอลัมน์
ALTER VIEW	แก้ไขวิวที่สร้างไว้ก่อนหน้านี้
CREATE DATABASE	สร้างฐานข้อมูลใหม่
CREATE PROCEDURE	สร้าง Stored Procedure ใหม่
CREATE SCHEMA	สร้าง Schema ขึ้นมาภายในฐานข้อมูล
CREATE TABLE	สร้างตารางใหม่ภายในฐานข้อมูล
CREATE VIEW	สร้างดาตาเบสวิวให้กับตาราง
DELETE	ลบแถวออกจากตาราง
DROP DATABASE	ยกเลิกฐานข้อมูลโดยการลบไฟล์ของมัน
DROP PROCEDURE	ยกเลิก Stored Procedure
DROP TABLE	ยกเลิกตารางออกจากฐานข้อมูล
DROP VIEW	ยกเลิกวิวออกจากฐานข้อมูล
INSERT	เพิ่มแถวใหม่เข้าไปในตารางหรือวิว
SELECT	เลือกฟิลด์ภายในตารางเพื่อนำมาดู
USE	ใช้ฐานข้อมูลที่กำหนด เป็นคำสั่งเริ่มต้นสำหรับคำสั่งต่อไป

2.5 Web Language

ก่อนที่เครื่องคอมพิวเตอร์สองเครื่องจะสามารถสนทนากันได้ พวกมันต้องได้รับการโปรแกรมไว้ให้พูดภาษาเดียวกันในปัจจุบันภาษาของเว็บมีอยู่มากมายและแต่ละภาษาก็มีจุดแข็งและจุดอ่อนของมันเอง ในขณะที่ภาษา HTML ก็ยังคงเป็นทางเลือกที่ดีที่สุดในขณะนี้ถ้าคุณต้องการภาษาที่ใช้งานง่ายๆ มีผลกระทบต่อระบบน้อยและตรงไปตรงมา แต่หากคุณต้องการภาษาที่มีความยืดหยุ่นในการโต้ตอบกับผู้ใช้ อพเทฐานข้อมูลแบบไดนามิกและสร้างกราฟิกที่สลับซับซ้อนได้แล้วละก็ ภาษาของจาวาถือว่าเป็นทางเลือกที่ดีที่สุด ไม่ว่าจะเป็นภาษาใด ในการทำความเข้าใจถึงวิธีการที่เว็บเซิร์ฟเวอร์ใช้ติดต่อสื่อสารกับไคลเอนต์บราวเซอร์บราวเซอร์ (และรวมถึงการค้นหาจุดอ่อนในการอิมพลิเมนต์ของมัน) จำเป็นต้องเข้าใจถึงเทคโนโลยีพื้นฐานที่รองรับก่อน

2.5.1 HTML

นามสกุลของไฟล์เป็น .html, .htm, .html4 HTML ย่อมาจาก Hypertext Markup Language ถูกสร้างขึ้นโดย Tim Berners-Lee ในปี ค.ศ. 1989 เป็นเฟรมเวิร์กพื้นฐานของอินเทอร์เน็ต เกือบๆทุกเว็บไซต์ในโลกล้วนแต่ใช้ภาษา HTML เพื่อแสดงผลข้อความ กราฟิก เสียง และภาพเคลื่อนไหว ผู้รับผิดชอบเกี่ยวกับมาตรฐานนี้ก็คือ World Wide Web Consortium (W3C) (<http://www.w3.org>)

ถึงแม้ภาษานี้จะใหม่ แต่มันถือกำเนิดมาจากภาษาเก่าแก่ที่ชื่อ Standard Generalized Markup Language (SGML) แรงผลักดันในการสร้างภาษา HTML ได้รับการพัฒนามาเกือบสิบปี และเริ่มเป็นรูปเป็นร่างจริงจังในช่วงที่ Bill Atkinson ได้พัฒนา HyperCard สำหรับเครื่องแมคอินทอชและระบบปฏิบัติการ MacOS สมมุติฐานเบื้องหลัง HyperCard ถือเป็นรากฐานสำคัญของภาษา HTML นั่นก็คือ การทำไฮเปอร์ลิงก์(hyper linking) ซึ่งก็คือ ความสามารถในการลิงค์หรือเชื่อมโยงจากคำๆ หนึ่งหรือพื้นที่หนึ่งของเพจไปยังอีกพื้นที่หรืออีกเพจหนึ่ง คอนเซ็ปต์นี้เป็นเรื่องง่าย แต่ก่อนที่จะมี HyperCard มันไม่เคยถูกนำมาใช้งานในเชิงปฏิบัติอย่างจริงจังเลย

HTML ประกอบไปด้วยชุดของ “องค์ประกอบย่อย (elements)” ซึ่งทำหน้าที่เป็นเสมือนภาษาที่คอยบอกของผู้ใช้แสดงผลสิ่งต่างๆ ตามต้องการบนหน้าจอ ในการทำงานของเรา เราได้ค้นพบว่าองค์ประกอบย่อยหรือ อิลิเมนต์ของ HTML ที่ง่ายๆ และดูเหมือนจะไม่น่าเป็นอันตรายนี้ สามารถถูกใช้เพื่อเข้าถึงเว็บเซิร์ฟเวอร์โดยไม่ได้รับอนุญาตได้ ตารางที่ 1.1 ประกอบด้วยลิสต์ของ HTML และผลกระทบด้านความปลอดภัย ดูข้อมูลเพิ่มเติมได้ที่ <http://www.w3.org/TR/html401>

ตารางที่ 2.3 แอตทริบิวต์และผลกระทบต่อความปลอดภัย

อิลิเมนต์/คุณลักษณะหรือแอตทริบิวต์	ผลกระทบด้านความปลอดภัย
<form>	เป็นฟอร์มสำหรับอินพุตข้อมูลจากผู้ใช้ เมื่อโปรแกรมรับอินพุตจากผู้ใช้ ความเสี่ยงด้านความปลอดภัยสำหรับเกิดขึ้นได้ นี่คือการส่วนใหญ่ที่แฮกเกอร์ใช้กัน นั่นก็คือ การส่งตัวอักขระที่โปรแกรมไม่คาดคิดว่าจะได้รับ ส่งผลให้เกิดผลลัพธ์ที่คาดไม่ถึง
<form action>	แอตทริบิวต์ Action ของ <form> เป็นตัวสร้างโปรแกรมที่สามารถเอ็กซีคิวต์ได้บนเว็บเซิร์ฟเวอร์ โดยการใช้ชื่อของโปรแกรมที่ทำหน้าที่ประมวลผลข้อมูลที่ผู้ใช้ส่งมาให้ ผู้บุกรุกสามารถเรียนรู้ข้อมูลที่มีประโยชน์เกี่ยวกับเว็บเซิร์ฟเวอร์และบางทีอาจค้นหาข้อมูลแบ็กอัพหรือเวอร์ชันเก่าของโปรแกรมในไดเรกทอรีเดียวกันหรือไดเรกทอรีอื่นๆ
<form method>	แอตทริบิวต์ Method ของ <form> นี้เป็นตัวสร้างกลไกสำหรับการส่งข้อมูลที่ใช้กรอกเข้ามาให้กับเว็บเซิร์ฟเวอร์ มีด้วยกันสองวิธีสำหรับการส่งข้อมูลไปยังโปรแกรมก็คือ POST, GET ด้วยการทำความเข้าใจวิธีการส่งข้อมูลนี้ แฮกเกอร์สามารถดักฟังข้อมูลหรือยิ่งไปกว่านั้น แก้ไขข้อมูลที่รับส่งกันได้และก่อให้เกิดผลที่ผิดปกติ
<script language=<variable>>	อิลิเมนต์ <script> เมื่อถูกใช้งานร่วมกับแอตทริบิวต์ “language” จะทำให้แฮกเกอร์สามารถแก้ไข Client-side scripting ที่กำลังถูกส่งมายังได้ เมื่อแฮกเกอร์สามารถแก้ไข Client-side script ได้ เขาก็จะสามารถเล็งผ่านสคริปต์สำหรับฟิวเตอร์หรือตรวจสอบเซ็คอินพุตได้ Client-side scripting ประกอบด้วย JavaScript ,VBScript, Iscript, XML
<input>	อิลิเมนต์ <input> ใช้เพื่อควบคุมอินพุตของฟอร์ม แอตทริบิวต์เฉพาะบางอย่างสามารถถูกแก้ไขเพื่อส่งข้อมูลแปรๆไปยังเว็บเซิร์ฟเวอร์
<input type=hidden>	แอตทริบิวต์ “type” เมื่อถูกกำหนดให้ค่าของมันเป็น “hidden” สามารถทำให้แฮกเกอร์เปลี่ยนแปลงแอตทริ

	บิวต์ “value” ให้เป็นบางสิ่งที่โปรแกรมไม่ต้องการ
<input maxlength=<variable>>	แอตทริบิวต์ “maxlength” สามารถถูกแก้ไขโดยแฮกเกอร์ได้ เพื่อให้คนมีโอกาสส่งสร้างขนาดใหญ่ที่สามารถดิสเอบิลการทำงานของเว็บเซิร์ฟเวอร์ได้ ถ้าเซิร์ฟเวอร์ไม่ได้มีประมวลผลในเบื้องต้นที่ดีพอ
<input size=<variable>>	แอตทริบิวต์ Size คล้ายกับแอตทริบิวต์ “maxlength” ตรงที่ว่า แอตทริบิวต์ “size” สามารถถูกแก้ไขได้โดยแฮกเกอร์เพื่อให้คนมีโอกาสส่งสร้างขนาดใหญ่ที่สามารถดิสเอบิลการทำงานของเว็บเซิร์ฟเวอร์ได้ ถ้าเซิร์ฟเวอร์ไม่ได้มีการประมวลผลในเบื้องต้นที่ดีพอ
<applet>	Java Applet อิลิเมนต์นี้ถูกใช้เพื่อแสดงผลหรือรันจาวาแอปเพล็ต เพราะว่าจาวาถูกส่งไปในลักษณะเคลียร์เท็กซ์และใช้งานบาร์โค้ดเพื่อการเอ็ชชีควิต มันจึงถูกลักลอบแท็บได้โดยใช้เครื่องมือวิเคราะห์โปรโตคอล เช่น Snort หรือ Ether Peek
<object>	อิลิเมนต์นี้โดยทั่วไปจะถูกใช้สำหรับการแสดงผล ActiveX Control แต่มันยังสามารถถูกใช้กับจาวาแอปเพล็ตก็ได้ แฮกเกอร์สามารถส่งอีเมลที่มีแท็ก HTML ฝังตัวอยู่และทำให้ผู้อ่านเมลเอ็ชชีควิต ActiveX Control ได้โดยไม่รู้ตัวเพื่อให้คนเข้าควบคุมระบบได้ อิลิเมนต์ <object>เป็นหนึ่งในหลายๆ วิธีที่ดีที่สุดสำหรับการเผยแพร่อีเมลไวรัส
<embed>	อิลิเมนต์นี้โดยทั่วไปจะถูกใช้ร่วมกับแท็ก <object> เพื่อแสดงผล ActiveX control และ Netscape plug-ins

2.5.2 PHP

นามสกุลของไฟล์ .php, .php3 และบางที่ไม่มีนามสกุล ถึงแม้จะมีผู้พัฒนาภาษา PHP อยู่มากมาย ผู้พัฒนาดั้งเดิมของ PHP ก็คือ Rasmus Lerdorf เขาได้เริ่มพัฒนาเอนจินสำหรับแปลภาษาของ PHP ในปี ค.ศ. 1995 โดยพัฒนาออกแบบในรูปแบบของโปรแกรมหนึ่งที่ทำงานภายใต้ Perl CGI ซึ่งเขาเองเรียกมันว่า “Personal Home Page” หรือเรียกง่ายๆ คือ PHP จุดประสงค์ดั้งเดิมของเขาก็เพื่อเก็บบันทึกสื่อกระจายละเอียดของผู้เข้าเยี่ยมชมเพื่อประวัติส่วนตัวของเขาบนเว็บ จากนั้นเขาได้พัฒนาทุกอย่างอย่างขึ้นมาใหม่ใน C และทำให้มันมีขนาดใหญ่ขึ้น เพิ่มขยายประสิทธิภาพของ

โปรแกรมด้วยความสามารถในการแปลภาษาที่หลากหลาย และความสามารถในการเชื่อมต่อเข้าสู่ฐานข้อมูลเป็นเวลากว่าหลายปีมาแล้ว มีโปรแกรมเมอร์หลายท่านช่วยกันพัฒนา PHP เช่น Zeev Suraski และ Andi Gutmans ผู้ซึ่งปรับปรุงเอนจินสำหรับแปลภาษาใหม่ให้เป็น PHP เวอร์ชัน 3

นอกเหนือจาก Active Server Pages (ASP) และ Perl แล้ว PHP ถือได้ว่าเป็นหนึ่งในภาษาสำหรับเขียน Server-side Script ที่ได้รับความนิยมมากตัวหนึ่งในปัจจุบัน แทบไม่น่าเชื่อว่าถ้าฟังภาษา PHP นั้นมีหลากหลายมากและสามารถนำไปพัฒนาแอปพลิเคชันแบบ standalone ที่ไม่เกี่ยวกับเว็บ อย่างไรก็ตาม ภาษานี้มักถูกใช้บ่อยครั้งบนเว็บเซิร์ฟเวอร์ที่รันบนยูนิกซ์ (โดยทั่วไปมักเป็น Apache ให้อูที่ <http://www.apache.org>) เพื่อทำหน้าที่เป็นเอนจินประมวลผลในฝั่งเซิร์ฟเวอร์จริงๆ แล้วมันเป็นโมดูลหนึ่งของ Apache ที่ถูกเรียกใช้บ่อยครั้งที่สุด

(http://www.securityspace.com/s_survey/data/man/200111/apachemods.html) ไฟล์ PHP สามารถถูกเปลี่ยนชื่อเป็นอะไรก็ได้ แต่โดยทั่วไปมักมีนามสกุลเป็น .php, .php3 เพื่อให้คุ้นเคยกับโค้ดของ PHP เราจะนำเสนอโค้ดตัวอย่างต่อไปนี้สักเล็กน้อย ซึ่งเป็นการใช้คำสั่ง print ของ PHP เพื่อแสดงผลสตริง “Hell World” ในเว็บเบราว์เซอร์ของผู้ใช้

(2.3)

```
<!-- PHP Example in HTML
<!-- Prints "PHP Example: Hello World!" to the browser
<html><head><title> PHP Example</title></head>
<?php
print "<br><h1>Hello World!<br></h1>";
?>
</html>
```

ให้สังเกตว่า PHP นั้นเหมือนกันมากกับ Perl ในแง่ที่ว่าสามารถใช้งานภาษาควบคู่ไปกับแท็ก HTML ได้ บรรทัดที่ 1 และ 2 คือคอมเม้นท์ของ HTML โดยสังเกตได้จากแท็ก <!-- บรรทัดที่ 3 นั้นเป็นหัวของแท็ก HTML ได้แก่ <html>, <head> และแท็ก <title> บรรทัดที่ 4, 5 และ 6 เป็นโค้ดของ PHP บรรทัดที่ 4 นั้นเป็นจุดเริ่มต้นของโค้ด PHP ดังสังเกตได้จากวงเล็บเปิด (<?> บรรทัดที่ 5 เป็นโค้ดของ PHP สำหรับการส่งสตริง “Hello World” ออกไปเป็นเอาต์พุตที่หน้าจอ บรรทัดที่ 6 เป็นวงเล็บปิด (?>) ที่คู่กับวงเล็บเปิดในบรรทัดที่ 4 ต่อไปนี้เป็นตัวอย่างอย่างง่ายของโค้ด PHP ซึ่งจะช่วยให้คุณคุ้นเคยกับเทคโนโลยีที่ได้รับความนิยมบนเว็บไซต์

(2.4)

```
<?
// Open the SQL connection
$conn = mysql_connect("10.1.1.1", "sa", "guessme") or die(mysqlerror());
```

```

@mysql_select_db("inventory") or die(mysqlerror());
// SQL query
$data = mysql_query("SELECT * FROM autos") or die(mysqlerror());
// Print the data in HTML
print "<table>\n";
while ($row = mysql_fetch_row ($data))
{
    print "<tr>\n";
    print "<td>$row[0]</td>\n";
    print "<td>$row[1]</td>\n";
    print "</tr>\n";
}
print "</table>\n";
// Close the SQL connection
mysql_close($conn);
?>

```

ข้อสังเกตที่เด่นชัดของ PHP ก็คือจุดอ่อนของมันนั้นคล้ายกันกับ Perl กล่าวคือถ้าสคริปต์ต้องประมวลผลอินพุตจากเว็บเบราว์เซอร์เพื่อติดต่อกับฐานข้อมูลหรือฟังก์ชัน system(), passthru(), shell_exec(), exec() หรือ server side include (SSI) แสกเกอร์สามารถใช้ประโยชน์จากการตรวจเช็คอินพุตที่ไม่ดีพอทำให้เอนจินของ PHP ประพฤติตัวไม่ดี อย่างเช่น เปิดโอกาสให้แฮกเกอร์คิดค้นสิ่งใดๆ ก็ได้ เพื่อลดความเสี่ยงดังกล่าวนี้ให้แน่ใจว่าได้รวมเอาฟังก์ชันการตรวจเช็คอินพุตที่ละเอียดเข้าไปในโปรแกรมของคุณแล้ว และเช่นเดียวกับ Perl คุณสามารถ (และควร) ใช้นิพจน์ปกติในโค้ด PHP เพื่อค้นหาข้อมูลแฝงตัวอันตรายที่อาจมีอยู่ในฟิลด์ที่รับค่าอินพุตจากผู้ใช้งานและแจ้งเตือนทันทีที่ค้นพบ โค้ดต่อไปนี้จะใช้ฟังก์ชัน preg_match() เพื่อเปรียบเทียบค่าตัวแปร \$string ที่ได้รับกับตัวอักขระปกติและสามารถถูกใช้เพื่อตรวจเช็คฟิลด์อินพุตที่เป็นตัวเลขได้ด้วย

(2.5)

```

if (preg_match("/^[0-9]+$/i", $string))
    echo "Error discovered in your number field\n";
    return 1;
break;

```


โค้ดต่อไปนี้อาจถูกใช้เพื่อตรวจสอบเช็คไฟล์ที่รับค่าอินพุตเป็นสตริงได้

(2.6)

```
if (preg_match("/^[a-z0-9]+$/" , $string))
    return 1;
break;
```

สำหรับข้อมูลเพิ่มเติมเกี่ยวกับPHPและสามารถดาวน์โหลดโปรแกรมให้เช็คว่า

<http://www.php.net>

2.5.3 จาวาสคริปต์

ภาษาจาวาสคริปต์สามารถทำงานได้อย่างเกือบสมบูรณ์บนเครื่องไคลเอนต์ ประโยชน์ของมันก็คือด้านความเร็วแต่ไม่ใช่เรื่องความปลอดภัย เพราะมันสามารถถูกแก้ไขโดยแฮกเกอร์เพื่อนำมาโจมตีได้

จาวาสคริปต์เป็นภาษาสคริปต์ที่อาศัยตัวตีความ (Interpreter) ไม่ได้อาศัยการคอมไพล์ (non-compiled) ซึ่งแต่ดั้งเดิมได้รับการพัฒนาขึ้นโดยเน็ตสเคป อย่างไรก็ตาม สิ่งที่เกี่ยวข้องกันระหว่างจาวาสคริปต์และจาวาก็คือ เพียงชื่อของมันเท่านั้นที่คล้ายกัน จาวาเป็นภาษาที่ต้องผ่านการคอมไพล์และเป็นลักษณะการโปรแกรมเชิงวัตถุ (object-oriented) และบางครั้งก็ยากที่จะทำความเข้าใจ ส่วนจาวาสคริปต์นั้นเป็นภาษาสคริปต์แบบง่ายๆ และมีลักษณะกึ่งๆ เชิงวัตถุ ซึ่งเหมาะสำหรับการพัฒนาเว็บอย่างรวดเร็ว ถึงแม้จาวาสคริปต์จะมีความสามารถในการเชิงวัตถุอย่างเช่น การตรวจเช็คข้อมูลที่ได้รับมาจากฟอร์ม การเพิ่มโค้ด HTML เข้าไปโดยอัตโนมัติตามความต้องการ และการคำนวณเฉพาะอย่างเช่น การคำนวณวันเวลาและการค้นหาประเภทของบราวเซอร์ ตัวอย่างง่ายๆ ของจาวาสคริปต์ก็คือโค้ดต่อไปนี้อซึ่งทำการแสดงไดอะล็อกบ็อกซ์เมื่อถูกคลิก

(2.7)

```
<html>
<head>
<title>Simple JavaScript Example</title>
<script language="Javascript">
<!-- hide for JavaScript challenged browsers
function popup()
{
    alert("Hello and welcome world!");
}
// Done hiding -->
```

```

</script>
</head>

<h1 align=center>My JavaScript example</h1>

<div align=center>

<form>

<input type="button" value="Hello World Me!" onclick="popup()">

</form>

```

ตัวอย่างนี้เป็นตัวอย่างที่ดีสำหรับการทำความเข้าใจภาษาและวิธีการเรียกใช้มันและวิธีการจดจำมันในระหว่างการแสกเว็บของคุณ เพราะจาวาสคริปต์ข้างต้นนี้ทำงานอยู่ในฝั่งไคลเอนต์ แสกเกอร์สามารถหลบผ่านฟังก์ชันการตรวจเช็คอินพุตไปได้ และส่งข้อมูลที่ไม่มาตรฐานเข้ามาด้วยหวังว่าจะแครชแอปพลิเคชันของคุณได้หรือเลวร้ายไปกว่านั้นก็คือได้ข้อมูลความลับสำคัญ ในตัวอย่างนี้เราใช้จาวาสคริปต์เพื่อตรวจสอบความถูกต้องของอินพุตที่รับมาจากผู้ใช้งานทางฟอร์มเพื่อให้แน่ใจว่าอายุที่ถูกต้องได้ถูกส่งลงไปในฟิลด์ Age ของฟอร์ม โค้ดจาวาสคริปต์ดังกล่าวได้จำกัดให้อายุที่ใส่เข้าไปมีค่าตั้งแต่ 1-110 ส่วนค่าอื่นๆ นอกเหนือจากนี้จะไม่ได้รับอนุญาต

(2.8)

```

<html>
<head>
<title>Validate User Input in our Form</title>
<script>
<!-- hiding again
function validate_input()
{
  if (document.testform.age.value < 1)
  {
    alert('Nice try young thing. Please enter your REAL age');
    return false;
  }
  else
  {
    if (document.testform.age.value > 110)
    {

```

```

        alert('Nice try granpa. Please enter your REAL age');
        return false;
    }
    else
    {
        alert('Thanks for being '+document.testform.age.value);
        return true;
    }
}
}
// done hiding -->
</script>
</head>
<body>
<form name=testform action="scripts/input.pl" method=post onSubmit="return
validate_input()">
    <b>Please enter your age:</b>
    <input type=text name=age size=3 maxlength=3><br><br>
    <input type=submit value="Submit Age">
</form>
</body>
</html>

```

ด้วยความรู้ที่ว่าจาวาสคริปต์ได้รับการประมวลผลโดยบราวเซอร์ที่ไคลเอนต์ เราสามารถเซฟไฟล์ HTML จากเว็บเซิร์ฟเวอร์และยกเลิกฟังก์ชันตรวจสอบเช็คอินพุตได้ ดังนั้นฟังก์ชัน validate_input() ในไฟล์ HTML ก็จะส่งค่า “จริง” กลับไปให้ทันทีเพื่อให้สมบูรณ์ จำเป็นต้องเปลี่ยนแอตทริบิวต์ action จากเดิมที่เป็นโลคอลเป็นการรีโมตแทนเพื่อที่จะส่งไปรันบนเซิร์ฟเวอร์

(2.9)

```

function validate_input()
{
    return true;
}
action="http://www.example.com/scripts/input.pl":

```

```
<form name=testform action="http://www.example.com/scripts/input.pl" method=post
onSubmit="return validate_input()">
```

คุณสามารถเปิดโลคอลไฟล์ และส่งค่าใดๆ ก็ได้ในฟิลด์ Age ตามที่คุณต้องการ และคลิกปุ่ม Submit ผลลัพธ์ที่ได้อาจไม่มีอะไรเกิดขึ้น หรืออาจทำให้เซิร์ฟเวอร์เคลชหรืออาจแสดงเมสเสจแจ้งข้อผิดพลาดที่เป็นประโยชน์ขึ้นมา

2.6 Uniform Resource Locator (URL)

URL คือกลไกหรือวิธีการในการระบุถึงทรัพยากรที่ต้องการบนเว็บ หรือบนเซิร์ฟเวอร์ FTP รวมทั้งโปรโตคอลในระดับแอปพลิเคชันเลเยอร์ ซึ่ง URL ดังกล่าวจะถูกส่งไปยังเว็บเซิร์ฟเวอร์ บางครั้ง URL ก็เป็นกลไกเดียวเท่านั้นสำหรับการสื่อสารกับระบบขนาดใหญ่ที่มีความซับซ้อนที่อยู่ด้านหลังไฟลวอลล์ที่ปลอดภัยที่สุด ในบรรดาเทคนิคนับไม่ถ้วนของการเจาะระบบการแฮกผ่านเว็บถือได้ว่าเป็นวิธีที่ง่ายและสะดวกที่สุด

หัวข้อที่จะศึกษาเกี่ยวกับ URL

- โครงสร้างของ URL
- URL encoding หรือ การเอ็นโค้ด URL
- รหัสแอสกีที่อยู่ในรูปแบบเลขฐานสิบหกและยูนิโค้ด (Unicode)
- ตัวอักขระเมตา (Meta-character)

2.6.1 โครงสร้างของ URL

โดยทั่วไป โครงสร้างพื้นฐานของ URL คือ

(2.10)

protocol://server/path/to/resource?parameters

แต่ละองค์ประกอบจะได้รับการอธิบายดังตารางดังนี้

ตารางที่ 2.4 องค์ประกอบของ URL

องค์ประกอบ	คำอธิบาย
Protocol	โปรโตคอลในระดับแอปพลิเคชันเลเยอร์ การใช้งานพื้นฐานส่วนใหญ่ของ URL ก็คือ การร้องขอทรัพยากรจากเว็บเซิร์ฟเวอร์ (HTTP servers) ดังนั้นโปรโตคอลพื้นฐานที่สุดก็คือ http: ส่วนโปรโตคอลอื่นอาจเป็น https:, ftp:, ldap:, telnet:, pop3: และอื่นๆ ขึ้นกับว่าบราวเซอร์และเซิร์ฟเวอร์สนับสนุนโปรโตคอลใดบ้าง
Server	ชื่อแบบ DNS, ชื่อแบบ NetBIOS หรือหมายเลข IP Address ของโฮสต์บนเน็ตเวิร์ก ซึ่งเก็บทรัพยากรที่กำลังถูกร้องขออยู่
Path/to/resource	ไคเรกทอรีพาธ รวมทั้งชื่อทรัพยากรของ (resource name) ของทรัพยากรที่ต้องการ ทรัพยากรที่ต้องการอาจเป็นได้ทั้งไฟล์สแตติกหรือแอปพลิเคชันที่สร้างเอาต์พุตอย่างไดนามิก
Parameters	(เป็นออปชันเสริม) พารามิเตอร์อาจถูกส่งผ่านเข้าไปยังทรัพยากร (resource) ถ้าทรัพยากรนั้นคือแอปพลิเคชันหรือโปรแกรมที่สร้างเอาต์พุตอย่างไดนามิก บางครั้ง ส่วนนี้ของ URL ซึ่งระบุพารามิเตอร์ยังถูกเรียกว่า Query String

http://www.blueballoon.com/pictures/davinci/monalisa.html

Protocol Server Name Path to File Being Requested

(a)

ftp://192.168.17.33/pub/img_viewer.exe

Protocol Server IP Address Path to File Being Requested

(b)

https://www.blueballoon.com/order/buy.asp?item=A003&pmt=visa

Protocol Server Name Path to Application Being Invoked Parameters Being Passed to the Application "buy.asp"

(c)

รูปที่ 2.6 ตัวอย่างของ URL

ในตัวอย่าง (a) นั้นอธิบายด้วยตนเองอยู่แล้ว ไฟล์ monalisa.html ซึ่งอยู่บนเซิร์ฟเวอร์ www.blueballoon.com กำลังได้รับการร้องขอผ่านทางโปรโตคอล HTTP ตำแหน่งของไฟล์ monalisa.html บนเว็บไซต์ www.blueballoon.com คือไดเรกทอรี /pictures/devinci

ในตัวอย่าง (b) เป็นตัวอย่างของการระบุโปรโตคอลอื่นแทนที่ HTTP โปรโตคอล FTP ที่ระบุนี้จะทำให้เบราว์เซอร์เปิดคอนเนกชันไปยังเซิร์ฟเวอร์ FTP ชื่อ [www.blueballoon.com](ftp://www.blueballoon.com) แบบ anonymous และดาวน์โหลดไฟล์ img_viewer.exe ที่เก็บในไดเรกทอรี /pup/

ในตัวอย่าง (c) เป็น URL ที่ชี้ไปเพื่อเรียกแอปพลิเคชันขึ้นมาทำงาน แอปพลิเคชันในที่นี้คือ buy.asp และเก็บอยู่ในไดเรกทอรี /order/ โดยมีพารามิเตอร์อยู่สองตัวที่ถูกส่งผ่านเข้าไปยังแอปพลิเคชัน ชื่อ "item" พร้อมด้วยค่า (value) ที่เท่ากับ "A003" และ "pmt" พร้อมด้วยค่า (value) ที่เท่ากับ "visa" สังเกตว่าโปรโตคอลไม่ใช่ "http:" แต่เป็น "https:" ซึ่งคือ HTTP ที่วิ่งอยู่บน Secure Socket Layer (SSL)

2.6.2 URLs และการส่งผ่านพารามิเตอร์

คิวรีสตริง (Query String) ของ URLs ถูกใช้เพื่อส่งผ่านพารามิเตอร์ไปยังแอปพลิเคชันที่กำลังถูกเรียกขึ้นมาทำงาน เมื่อแอปพลิเคชันดังกล่าวถูกเรียกโดยเว็บเซิร์ฟเวอร์ แอปพลิเคชันนั้นจะรับสองสิ่งมาจากเว็บเซิร์ฟเวอร์นั่นก็คือ ตัวแปรสถานะแวดล้อมของระบบและพารามิเตอร์ของ

โปรแกรม วิธีการที่เว็บแอปพลิเคชันจะโต้ตอบกับเว็บเซิร์ฟเวอร์ได้รับการอธิบายไว้ในข้อกำหนด Common Gateway Interface (CGI) ข้อกำหนดดังกล่าวจะกำหนดว่าแอปพลิเคชันโปรแกรมที่ถูกเรียกโดยเว็บเซิร์ฟเวอร์จะรับคิวรีสตริง(Query String) ผ่านทางอาร์กิวเมนต์ของบรรทัดคำสั่งและผ่านทางตัวแปรระบบชื่อ QUERY_STRING

ตัวอย่างที่อธิบายถึงวิธีการส่งพารามิเตอร์เข้ามายังแอปพลิเคชันโดยการสร้างสคริปต์ CGI ที่ชื่อ query.cgi บนเซิร์ฟเวอร์ลินุกซ์ซึ่งมี IP Address เป็น 192.168.7.253 โค้ดภายในของ query.cgi ดังข้างล่าง

(2.11)

```
01: #!/bin/sh
02: set -f
03: echo Content-type: text/plain
04: echo
05: echo Number of command-line args: "$#"
06: echo command-line args: "$*"
07: echo GATEWAY_INTERFACE = "$GATEWAY_INTERFACE"
08: echo SERVER_PROTOCOL = "$SERVER_PROTOCOL"
09: echo REQUEST_METHOD = "$REQUEST_METHOD"
10: echo SCRIPT_NAME = "$SCRIPT_NAME"
11: echo QUERY_STRING = "$QUERY_STRING"
```

สคริปต์นี้จะพิมพ์จำนวนของอาร์กิวเมนต์ที่ส่งให้มัน ค่าจริงๆของอาร์กิวเมนต์และตัวแปรสถานะแวดล้อมของระบบเล็กน้อยซึ่งถูกเซตโดยเว็บเซิร์ฟเวอร์เมื่อสคริปต์ถูกเรียกเช่น

`http://192.168.7.253/cgi-bin/query.cgi?Hello+World,+this+is+CGI`

คิวรีสตริง (query string) คือ “Hello+World+this+is+CGI ส่วนผลลัพธ์หรือเอาต์พุตของสคริปต์นี้คือ

(2.12)

```
Number of command-line args: 5
command-line args: Hello World, this is CGI
GATEWAY_INTERFACE = CGI/1.1
SERVER_PROTOCOL = HTTP/1.0
REQUEST_METHOD = GET
```

```
SCRIPT_NAME = /cgi-bin/query.cgi
QUERY_STRING = Hello+World,+this+is+CGI
```

เว็บเซิร์ฟเวอร์จะบรรจุคิวรีสตริง (query string) ไว้ในตัวแปรระบบที่ชื่อ QUERY_STRING มันจะดึงเอาพารามิเตอร์ที่อยู่ในสตริงออกมาและส่งเป็นอาร์กิวเมนต์เข้าไปยังสคริปต์ query.cgi สังเกตว่าเครื่องหมาย + ถูกแทนที่ด้วยช่องว่างโดยเว็บเซิร์ฟเวอร์

<http://192.168.7.253/cgi-bin/query.cgi?item=A003&pmt=visa>

ผลลัพธ์ที่แสดงบนบราวเซอร์ของไคลเอนต์

(2.13)

```
Number of command-line args: 0
command-line args:
GATEWAY_INTERFACE = CGI/1.1
SERVER_PROTOCOL = HTTP/1.0
REQUEST_METHOD = GET
SCRIPT_NAME = /cgi-bin/query.cgi
QUERY_STRING = item=A003&pmt=visa
```

ขณะนี้ เว็บเซิร์ฟเวอร์จะไม่ส่งค่าคิวรีสตริงเข้าไปเป็นอาร์กิวเมนต์ อย่างไรก็ตาม ตัวแปร QUERY_STRING จะเก็บเนื้อหาภายในคิวรีสตริง (query string) ข้อแตกต่างก็คือ ในตัวอย่างนี้คิวรีสตริงจะถูกสร้างขึ้นด้วยวิธีการส่งพารามิเตอร์มาตรฐานของ URL ซึ่งยึดถือตามข้อกำหนดของวิธีการส่งหลายๆ พารามิเตอร์พร้อมด้วยค่าของมันเข้าไปยังเว็บแอปพลิเคชัน ฟอรัมมาตรฐานของการส่งพารามิเตอร์ผ่านทางคิวรีสตริง (query string) ก็คือ

(2.14)

```
http://server/app\_program?param\_name1=value1&param\_name2=value2&...param\_nameN=valueN
```

ถ้า 3 พารามิเตอร์ถูกส่งผ่านเข้าไปยังแอปพลิเคชันโปรแกรม ชื่อของ 3 พารามิเตอร์พร้อมด้วยค่าของมัน ซึ่งถูกเชื่อมรวมกันด้วยตัวอักษร & จะถูกใส่ไว้ในคิวรีสตริง (query string) จากนั้นแอปพลิเคชันจะสกัดเอาพารามิเตอร์และค่าของมันออกมาอีกทีหนึ่งจากคิวรีสตริงที่ส่งมาให้

2.6.3 การเ็นโค้ด URL (URL Encoding)

โดยตัวมันเองแล้ว URLs นั้นเป็นเพียงแค่ชุดสตริงที่ประกอบด้วยตัวเลข ตัวอักษรและตัวอักขระต่างๆ โดยมีสัญลักษณ์พิเศษบางอย่างอยู่ข้างบน ชุดของตัวอักขระที่สามารถบรรจุอยู่ใน URL ได้จะประกอบด้วยสิ่งต่อไปนี้

สัญลักษณ์	ค่า
สัญลักษณ์ของตัวเลข ตัวอักษรปกติ	A-Z, a-z, 0-9
สัญลักษณ์พิเศษที่สงวนไว้	; / ? : @ & = + \$, < > # % "
ตัวอักขระพิเศษอื่นๆ	- _ . ! ~ * ' () { } ^ [] `

สำหรับส่วนใหญ่ สตริง URL จะประกอบด้วยตัวอักขระ ตัวเลขและสัญลักษณ์พิเศษที่สงวนไว้ที่มีความหมายพิเศษภายในสตริง URL ส่วนตัวอักขระพิเศษอื่นๆ จะถูกพบอยู่ในบางสตริง URL ถึงแม้พวกมันจะไม่มีมีความหมายพิเศษภายใน URL โดยตรง อย่างไรก็ตาม พวกมันอาจมีความหมายพิเศษสำหรับเว็บเซิร์ฟเวอร์ที่กำลังรับ URL นั้นมาหรือสำหรับแอปพลิเคชันที่ถูกร้องขอให้ทำงานผ่านทางเว็บเบราว์เซอร์

ตารางที่ 2.5 ตัวอักขระพิเศษและความหมายของมันภายใน URL

ตัวอักขระพิเศษ	การตีความและความหมาย
?	เป็นตัวแบ่งแยกจุดเริ่มต้นของคิวรีสตริง สิ่งต่างๆ ที่อยู่ข้างขวาของสัญลักษณ์ ? คือคิวรีสตริง (query string)
&	เป็นตัวคั่นกลางระหว่างพารามิเตอร์ถูกใช้เพื่อแบ่งกันระหว่างคู่ลำดับของพารามิเตอร์ซึ่งประกอบด้วยชื่อและค่าของมัน (name=value) ซึ่งอยู่ในคิวรีสตริง
=	เป็นตัวคั่นกลางระหว่างชื่อของพารามิเตอร์กับค่าของมัน (name=value) ซึ่งใช้ในขณะที่มีการส่งผ่านพารามิเตอร์ด้วยคิวรีสตริง
+	ถูกตีความว่าเป็นพื้นที่ว่าง (space)
:	เป็นตัวบ่งชี้ประเภทของโปรโตคอล ส่วนหน้าของ URL ที่เริ่มนับจากจุดเริ่มต้นจนถึงสัญลักษณ์ : จะระบุถึงโปรโตคอลในระดับแอปพลิเคชันเลเยอร์ที่ใช้ในการร้องขอทรัพยากร
#	ใช้เพื่อระบุจุดภายในเว็บเพจ ตัวอย่างเช่น URL http://www.acme-art.com/index.html#gallery และ http://www.acme-art.com/index.html#prurchase จะนำพาคุณไปสู่ตำแหน่งที่แตกต่างกันสองที่

	ภายในเว็บเพจเดียวกัน คือ เพจ index.html
%	ถูกใช้เป็นตัวกั้นสำหรับการระบุถึงตัวอักขระที่ถูกแปลงฟอร์แมตให้อยู่ในรูปแบบเลขฐานสิบหก
@	ถูกใช้ใน mailto: URL ในขณะระบุอีเมลแอคเดรสบนอินเทอร์เน็ตหรือในการส่งชื้อลือกอนของผู้ใช้เข้าไปยังทรัพยากรที่ต้องการตรวจสอบลือกอนของผู้ใช้ก่อน โดยเฉพาะเมื่อติดต่อกับเซิร์ฟเวอร์ FTP
~	ถูกใช้เพื่อระบุโฮมไดเรกทอรีของผู้ใช้บนเครื่องที่รองรับผู้ใช้หลายคน อย่างเช่น ยูนิคซ์ URL จะดูคล้าย http://server/~user_login_id/

2.6.4 ตัวอักขระเมต้า (Meta-characters)

ตัวอักขระอย่างเช่น * ; | ' มีความหมายพิเศษในฐานะที่เป็นตัวอักขระเมต้า (meta-character) ในแอปพลิเคชันหรือสคริปต์ ตัวอักขระเหล่านี้ไม่มีผลกระทบต่อ URL ไม่ว่ากรณีใด แต่ถ้าพวกมันบางตัวหรือหลายตัวถูกส่งผ่านเข้าไปยังแอปพลิเคชันได้ด้วยวิธีเฉพาะของมัน มันอาจก่อให้เกิดความหมายพิเศษที่คาดไม่ถึงได้และบางครั้งก็กลับกลายเป็นการสร้างช่องโหว่ด้านความปลอดภัยให้เกิดขึ้น ตัวอักขระเมต้า หลายตัวนั้นถูกตีความหมายแตกต่างกันไปโดยเว็บเซิร์ฟเวอร์ต่างยี่ห้อกัน ตัวอักขระเมต้า และ Input Validation สาเหตุที่พบบ่อยมากที่สุดกว่า 90% ของช่องโหว่ในเว็บแอปพลิเคชันทั้งหมดก็คือ การขาดการตรวจเช็คอินพุต

2.6.5 การระบุตัวอักขระพิเศษบนสตริง URL

ข้อกำหนดของ URL เปิดโอกาสให้เราได้เขียนตัวอักขระหรือสัญลักษณ์พิเศษเหล่านั้นในรูปแบบของเลขฐานสิบหกที่ตีความออกมาแล้วเท่ากับรหัสแอสกีของสัญลักษณ์พิเศษนั้น โดยขึ้นต้นด้วยสัญลักษณ์ % ดังต่อไปนี้

ตัวอักขระ	ค่าของเลขฐานสิบหก
ตัวอักขระทั่วไปทั้งหมด	%XX (%00-%FF)
ตัวอักขระควบคุม	%00-%1F, %7F
ตัวอักขระแอสกี	8 บิต %80-%FF
Spacebar	%20 หรือ +
Carriage Return (CR)	%0d
Line feed (LF)	%0a

2.6.6 การเ็นโค้ดด้วยฟอร์แมตของยูนิโค้ด (Unicode Encoding)

ถึงแม้การเ็นโค้ดตัวอักษรแอสกีด้วยเลขฐานสิบหกจะรองรับการใช้งานได้หลากหลายก็ตาม แต่มันก็ยังไม่กว้างพอที่จะถูกใช้แทนสัญลักษณ์ที่มากกว่า 256 ตัวได้ ระบบปฏิบัติการส่วนใหญ่ในปัจจุบันและแอปพลิเคชันส่วนใหญ่สามารถรองรับการนำเสนอเซตของตัวอักษรในลักษณะมัลติไบต์ได้จึงทำให้รองรับภาษาอื่นได้มากมายนอกจากภาษาอังกฤษ IIS Web Server ของไมโครซอฟต์สามารถรองรับ URLs ที่ประกอบด้วยตัวอักษรซึ่งถูกเ็นโค้ดไว้ด้วยฟอร์แมตแบบมัลติไบต์ของ UCS Translation Format (UTF-8) นอกเหนือจากการเ็นโค้ดแบบแอสกีธรรมดา ยูนิโค้ดมองในมุมของ URL ตัวอักษรยูนิโค้ดแบบสองไบต์นั้นได้รับการเ็นโค้ดหรือเข้ารหัสไว้โดยการใช้ %uXXYY โดยที่ XX และ YY คือค่าแบบเลขฐานสิบหกของตัวอักษรในไบต์สูงและไบต์ต่ำตามลำดับ สำหรับตัวอักษรแอสกีมาตรฐานที่เป็น %00 ถึง %FF นั้นจะได้รับการแปลงให้อยู่ในฟอร์แมตของยูนิโค้ดเป็น %u0000 ถึง %u00FF เว็บเซิร์ฟเวอร์จะทำการตีความในแบบ 16 บิตทุกครั้งเมื่อต้องใช้งานตัวอักษรแบบยูนิโค้ด